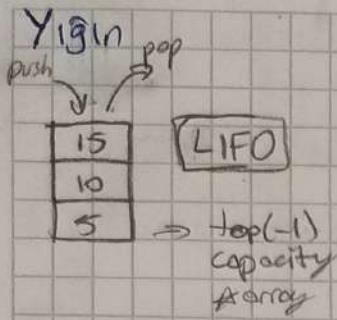


LIFO = Last in First Out



- Uygulama Alanları
- 1) Sembol dergisi
 - 2) Infix, postfix, prefix
 - 3) Redo-Undo
 - 4) Web browser
 - 5) Hanoi Tower Tree
 - 6) Sudoku

infix prefix (soldan)
 $A+B \Rightarrow +AB$
 $\Rightarrow AB+$
 postfix (soldan)

```
struct stack {
    int top;
    unsigned capacity;
    int *array;
}
```

Böyle belli diziye dizilimlenmek için yitirilebilir.

```
int isFull(struct stack *x) {
    return x->top == x->capacity - 1;
}
```

```
void push(struct stack *x, int y) {
    if (isFull(x) == 1)
        return;
    x->array[x->top] = y;
}
```

```
int pop(struct stack *x) {
    if (isEmpty(x))
        return;
    int alinan = x->array[x->top];
    x->top = x->top - 1;
    return alinan;
}
```

```
struct stack *createStack(unsigned capacity) {
    struct stack *yeni = (struct stack *) malloc(sizeof(struct stack));
    yeni->capacity = capacity;
    yeni->top = -1;
    yeni->array = (int *) malloc(sizeof(int) * yeni->capacity);
    return yeni;
}
```

```
int isEmpty(struct stack *x) {
    return x->top == -1;
}
```

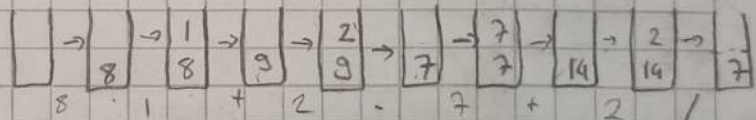
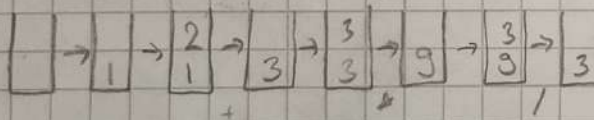
```
void fun(int n) {
    stack s;
    while (n > 0)
        push(&s, n % 2);
    while (!isEmpty(s))
        printf("%d", pop(&s));
}
```

n=10	1
n=5	0
n=2	1
n=1	0

Girilen sayı, 2'lik sisteme dönüştürülür

postfix $\Rightarrow 12+3*3/$

prefix $\Rightarrow /2+7-2+18$



Postfix Değerlendirme:

- 1) Sıradaki elemanı oku.
- 2) Eleman operand ise:
 - i) Push(operand)
- 3) Eleman operator ise:
 - i) Pop(s) yap (2 kez)
 - ii) İşlemi değerlendir.
 - iii) Sonucu push yap.

Input: A+B*(C+D)
 Output: ABC+D+
 Input: (A+B)-(C*(D/E))+F
 Output: AB+(CDE)/F+

$x \rightarrow array[x \rightarrow top] = y$

$yeni \rightarrow array = (int *) malloc(sizeof(int) * yeni \rightarrow capacity);$

Date:

Subject:

Infix to Postfix

- 1) Sadece sağ tara.
- 2) Eğer terimlerin elemanı operatöre ise postfix ifadeye yaz.
- 3) Diğer durumlarda;
Artarak önceliği yığın altına yazarsak,
veya yığın boşsa veya yığın '(' karşısına
yığına operatöre bas.
değilse
operatör al yığından işlem yap.
- 4) '(' gelirse yığına bas.
- 5) ')' gelirse yığına bas.
- 6) 2-5 adımlarını tekrar et.
- 7) Terimlerin bittiğinde pop ile bitir.

Input: $((A+B) - C * (D/E)) + F$

Input: $((A * B) / (D - C) + E / F)$

<u>Prefix</u>	<u>Infix</u>	<u>Postfix</u>
(	(	-
(	((	-
A	((	A
+	((+	A
B	((+	AB
)	(	AB+
-	(-	AB+
C	(-	AB+C
*	(-*	AB+C
(	(-*(	AB+C
D	(-*(	AB+CD
/	(-*/	AB+CD
E	(-*/	AB+CDE
)	(-*	AB+CDE/
)	(	AB+CDE*-
+	(+	AB+CDE*-
F	(+	AB+CDE*-F
)	-	AB+CDE*-F+

Output: AB + CDEA - F +

Output: $AB \rightarrow DC - EF / +$

Input: $8 \star 2 + (18 - 2) / 4 - 3$

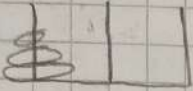
Termin	Yigin	Postfix	Termin	Yigin	Postfix
8	-	8	)	+ +	82*18,2-
*	*	8	/	+ /	82*18,2-
2	*	82	4	+ /	82*18,2-4
+	+	82*	-	+ -	82*18,2-4 /
(	+(	82*	3	+ -	82*18,2-4 / 3
	+(	82*1	)	-	82*18,2-4 / 3 -
8	+(	82*18			
-	+(	82*18			
2	+(	82*18,2			

Overett 18
Block of,

$\begin{array}{|c|} \hline 2 \\ \hline \end{array}$
 $\begin{array}{|c|} \hline 16 \\ \hline \end{array}$
 $\begin{array}{|c|} \hline 2 \\ \hline \end{array}$
 $\begin{array}{|c|} \hline 16 \\ \hline \end{array}$
 $\begin{array}{|c|} \hline 4 \\ \hline \end{array}$
 $\begin{array}{|c|} \hline 16 \\ \hline \end{array}$
 $\begin{array}{|c|} \hline 9 \\ \hline \end{array}$
 $\begin{array}{|c|} \hline 16 \\ \hline \end{array}$
 $\begin{array}{|c|} \hline 4 \\ \hline \end{array}$
 $\begin{array}{|c|} \hline 16 \\ \hline \end{array}$
 $\begin{array}{|c|} \hline 16 \\ \hline \end{array}$

strlen(dizi) → dizi uzunlu

Horari Tower



en sağa geçirmek için
 $n = \text{disk sayısı}$
 $2^n - 1$ adet hamle gerekir.

Kod Kapsamına göre postfix ile işlem yapmanın

Kuyruk (Queue)

struct queue

```
int front, rear, size;  
unsigned capacity;  
int *array;
```

}

Bunu böyle kuyruğu düşünersek oluruz.

rear

capacity → o gün boşta olan toplam
kaç kişi gelecek.

front

iş biten gittiğe

front artar.

rear sağa yeni kişi eklen-

mediği sürece değişmez.
Kişi sayısı azaldıkça size
azalır.

Enqueue → Kuyruğa

Dequeue → Kuyruktan eleman alma

Front → izleme başı

rear → izleme sonu

isFull → Dolmuş kuyruk ona bakar → size = capacity durumu

Enqueue ile ilgili işlemler

1. Dolu mu değil mi?
2. Dolu ise ekle.
3. Dolu değilse rear arttır.
4. Rear noktasına eleman ekle.
5. İşlemi bitir.

*Kuyruğu bağlı liste ile
deneyin demiz

<time.h> → bu kodun ne kadar
Zaman aldığına bakarız. (ns cinsinden)

struct queue *createqueue(unsigned capacity)

```
struct queue *yeni = (struct queue *) malloc(sizeof(struct queue));  
yeni → capacity = yeni → front = 0;  
yeni → rear = yeni → capacity - 1;  
yeni → array = (int *) malloc(yeni → capacity * sizeof(int));  
return yeni;
```

}

```
int isFull(struct queue *x){  
return (x → capacity == x → size);  
}
```

```
int isEmpty(struct queue *x){  
return (x → size == 0);  
}
```

```
void enqueue(struct queue *x, int y){  
if (isFull(x))  
return;  
x → rear = (x → rear + 1) % x → capacity;  
x → array[x → rear] = y;  
x → size = x → size + 1;  
}
```

```
int dequeue(struct queue *x){  
if (isEmpty(x))  
return;  
int item = x → array[x → front];  
x → front = (x → front + 1) % x → capacity;  
x → size = x → size - 1;  
return item;  
}
```

veri tipi
yazma
farklı

Date:

char a = '8';
int x = atoi(a);

char *int
cast

Subject:

Struct Emre

Binary tree ve Binary search tree farklı
şeylerdir.

Özellikler
1) Her tip kuyruk veri
CPU ve Diskte
kullanılır,

Kuyruğun Avantajları

- 1) Çok büyük boyutlu verileri yönetme kolay
- 2) Ekleme - silme kolay
- 3) Birden fazla tüketicinin bir şeye erişmesi kolay
- 4) Görevler arası iletişim kolay
- 5) Diğer veri yapılarının oluşturulmasında kullanılır.

Sıralı işlem için kuyruk yığınca
iyi ama en son işlemlerin
yığın

Dezavantajları

- 1) Ortadan eleman silmek zor (Limitation)
- 2) Dizi yapıyla size belirtmek dezavantaj
- 3) Eleman arama yapmak zor.

```
int values[] = {1, 3, 5, 7, 9, 11, 13, 15, 17, 19}
```

```
stack<int> s;
```

```
for(int i=0; i<10; i++)
```

```
    s.push(values[i]);
```

```
int n=25;
```

```
for(int i=0; i<4; i++)
```

```
    n+=s.top(); s.pop();
```

```
}
```

```
for(int i=0; i<2; i++)
```

```
    n-=s.top(); s.pop();
```

```
}
```

```
cout << n << endl;
```

13
17
3
1

n=25

19
17
15
13
64

n=89

11
13
20

n=69

7
5
3
1

rear=6 size=7 front=0

printf("%d", median(x));
↳ kodu ne dönecek?

```
void median(struct queue *x)
```

```
{ if (isEmpty(x))
```

```
    return;
```

```
int variable=0, int ind1=x->front, int tmp;
```

```
for (int i=ind1; i<rear, i++)
```

```
    variable++;
```

```
for (i=0, i<variable-1, i++)
```

```
    for (j=i+1, j<variable, j++)
```

```
        if (x->array[i] < x->array[j])
```

```
            tmp = x->array[i];
```

```
            x->array[i] = x->array[j];
```

```
            x->array[j] = tmp;
```

```
}
```

```
}
```

```
ind1 = variable/2;
```

```
printf("%d", x->array[ind1]); // return döndür ki
```

```
struct queue {
```

```
    int rear, front, size;
```

```
    unsigned capacity;
```

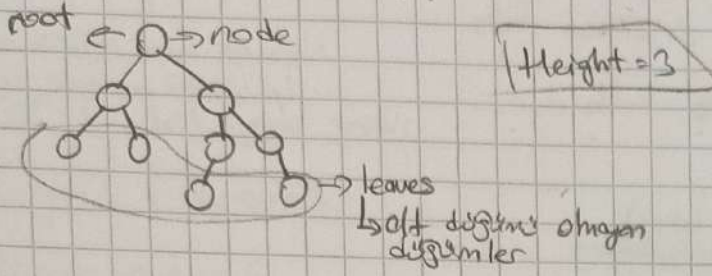
```
    int *array;
```

```
}
```

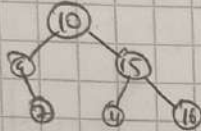
★ Binary tree ve binary search tree farklı ★

Binary Tree

Ağaç hiyerarşik veri yapısıdır.



10 16 7 11 4 15



Uygulama alanları

- 1) Hiyerarşik veri yönetim
- 2) Bilgi erişimi hızlandırma
- 3) Sıralı veri ile çalışırken
- 4) Dijital resimleri sıkıştırma için
- 5) Router algoritmaları
- 6) Gök aşaraki karar

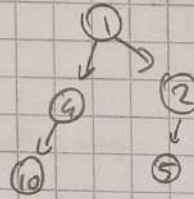
Neden ağaç

- 1) Dosya sisteminde hiyerarşik veri tutar.
- 2) Ortalama erişim/arama sağlar. (Bağlı listeden hızlı diziden yavaş)
- 3) Ortalama ekleme/silme sağlar.
- 4) Sınırsız eleman ekle.

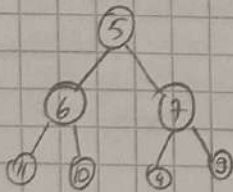
```
struct node {
    int data;
    struct node *left;
    struct node *right;
};
```

```
struct node *createNode(int x) {
    struct node *yeni = (struct node *) malloc (sizeof (struct node));
    yeni->data = x;
    yeni->left = yeni->right = NULL;
    return (yeni);
}
```

```
void main() {
    struct node *tree = createNode(1);
    tree->left = createNode(4);
    tree->left->left = createNode(10);
    tree->right = createNode(2);
    tree->right->left = createNode(5);
}
```

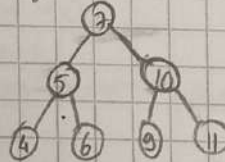


Binary Tree



Her node en fazla iki child node alabilir
olursa binary tree.

Binary Search Tree



Binary tree ama right child parentten büyük
left child parentten küçük

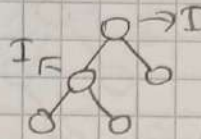
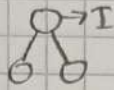
İkili Ağaç Türleri

Full Binary Tree

Tanım: Her düğümü 2 veya 0 (child) düğüme sahip ağaçtır.

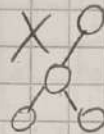
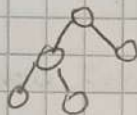
$$L = I + 1$$

Leaves Internal nodes



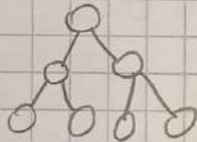
Complete Binary Tree

Tanım: Son seviye hariç tüm seviyeleri dolu olan ağaçtır.



Perfect Binary Tree

Tanım: Tüm iç değerlerin 2 adet alt düğümü vardır ve tüm yapraklar aynı seviyededir.

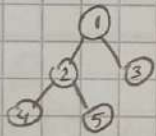


$$N = 2^n - 1$$

$n \rightarrow$ height
 $2^3 - 1 = 7$ düğüm

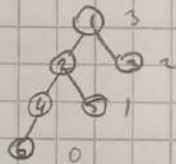
Balanced Binary Tree

$H = O(\log n)$ olan ağaçtır.
 $n =$ düğüm sayısı



$$\log_2 5$$

$2^2 \rightarrow H$
 $2 = 2$
Balanced



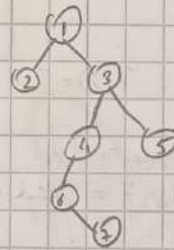
Balanced değil

$$3 \neq 2$$

$$\log_2 6 = x$$

$$2^x = 6$$

$x = 2$ değil



Balanced değil

$$\log_2 7 = x$$

$$2^x = 7$$

$x = 2$ değil

Ağaç yüksekliği bulma

```
int height(struct tnode *x) {
```

```
    if (x == NULL)
```

```
        return 0;
```

```
    int lheight = height(x->left);
```

```
    int rheight = height(x->right);
```

```
    if (lheight > rheight)
```

```
        return lheight + 1;
```

```
    else
```

```
        return rheight + 1;
```

} return (lheight > rheight) ? (lheight + 1) : (rheight + 1);
yazılabilir.

}

height(1)
height(2)
height(4)
height(NULL)

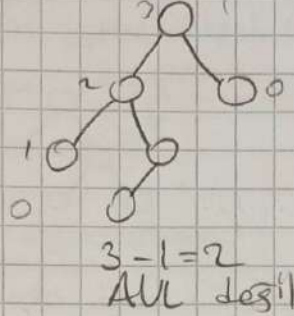
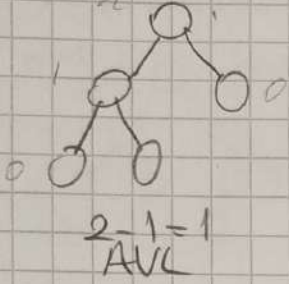
height(NULL)
return
height(1)
return
height(2)
return
height(4)
return

height(NULL)
return
height(1)
return
height(2)
return
height(4)
return

Subject:

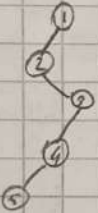
AVL

Tanım: İki alt ağac arasındaki yükseklik farkı max 1 olmalıdır.

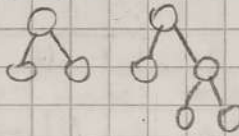


Pathological Tree

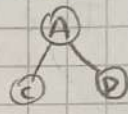
Tanım: Her iç düğümün 1 adet alt düğümü olmalıdır.



Unlabeled:



Labeled:

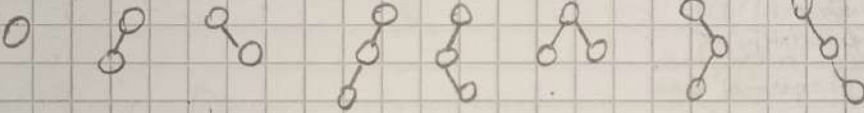


n=0

n=1

n=2

n=3



$T(0)=1$

$T(1)=1$

$T(2)=2$

$T(3)=5$

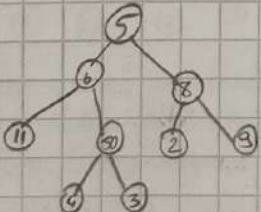
$$T(n) = \sum_{i=1}^n (i-1) \cdot T(n-i)$$

$$T(n) = \frac{2 \cdot n!}{(n+1)! \cdot n!}$$

Etkisiz düğümlerle kaç farklı ağac olur derse $n!$ ile çarpılır.

Etkisiz düğüm sayısı ile kaç farklı ağac, çarpılarak formülü

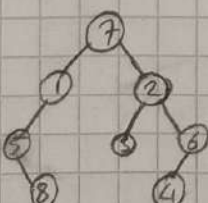
$$T(n) = \frac{2 \cdot n! \cdot n!}{(n+1)! \cdot n!}$$



In order $\Rightarrow$ sol, kök, sağ $\Rightarrow 11, 6, 4, 5, 0, 3, 8, 2, 9$

Pre order $\Rightarrow$ kök, sol, sağ $\Rightarrow 5, 6, 11, 5, 0, 4, 3, 8, 2, 9$

Post order $\Rightarrow$ sol, sağ, kök $\Rightarrow 11, 4, 3, 5, 0, 6, 2, 9, 8, 5$



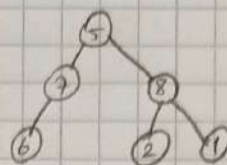
In order $\Rightarrow 5, 8, 1, 7, 3, 2, 4, 6$

Post order $\Rightarrow 8, 5, 1, 3, 4, 6, 2, 7$

Pre order $\Rightarrow 7, 1, 5, 8, 2, 3, 4, 6$

Inorder

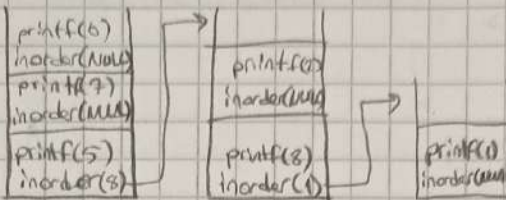
```
void inorder(struct node *x) {
    if (x == NULL)
        return;
    inorder(x->left);
    printf("%d\t", x->data);
    inorder(x->right);
}
```



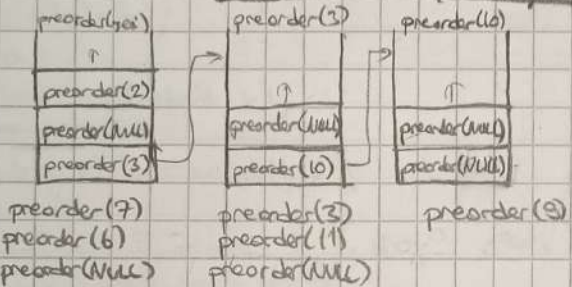
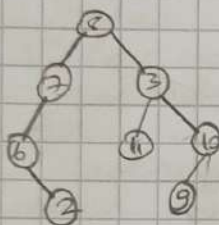
yeri = new Node(5)
yeri->left = new Node(7);
!

$\text{inorder}(\text{yeri}) \Rightarrow \text{inorder}(7) \Rightarrow \text{inorder}(6) \Rightarrow \text{inorder}(\text{NULL})$

```
void preorder(struct node *x) {
    if (x == NULL)
        return;
    printf("%d\t", x->data);
    preorder(x->left);
    preorder(x->right);
}
```

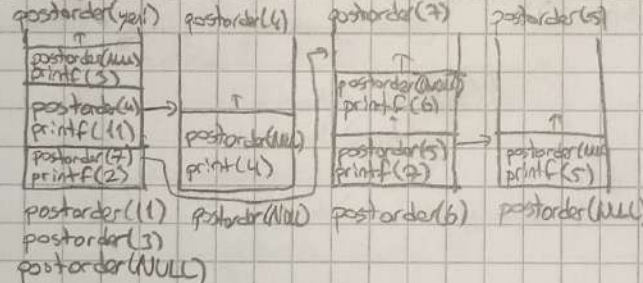
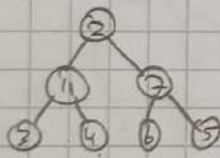


OUTPUT: 6, 7, 5, 2, 8, 1



OUTPUT: 8, 7, 6, 2, 3, 11, 10, 9

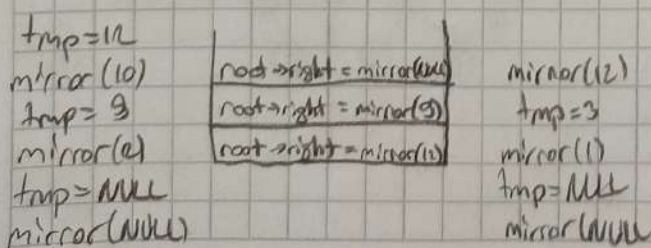
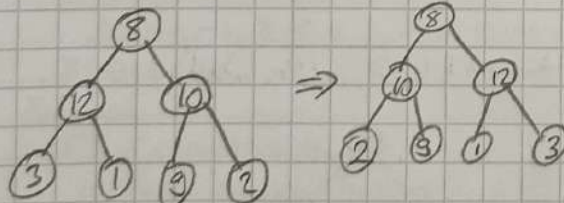
```
void postorder(struct node *x) {
    if (x == NULL)
        return;
    postorder(x->left);
    postorder(x->right);
    printf("%d\t", x->data);
}
```



OUTPUT: 3, 4, 11, 6, 5, 7, 2



```
struct node *mirror(struct node *root) {
    if (root == NULL)
        return NULL;
    struct node *tmp = root->left;
    root->left = mirror(root->right);
    root->right = mirror(tmp);
    return root;
}
```



mirror(5)
tmp=NULL
mirror(NULL)

Subject:

Date:

bir tar 2025
sorum

Agar silme

```
void sil(struct node *root)
{
    if (root == NULL)
        return;
    sil(root->left);
    sil(root->right);
    free(root);
}
```

statik deşiplexer

```
#include <stdio.h>
void foo() {
    int a=10;
    static int sa=10;
    a+=5;
    sa+=5;
    printf("%d", "%d\n", a, sa);
}
```

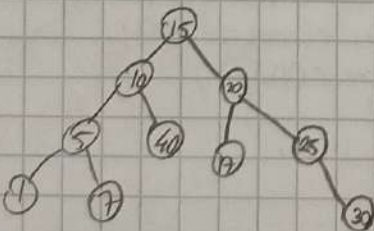
```
int main() {
    int i;
    for(i=0; i<40; ++i)
        foo();
}
```

ne iletir verici deşer sağa yoz.

Agar deşer sağa yoz

```
static int count=0;
int say(struct node *root) {
    if (root != NULL) {
        say(root->left);
        count++;
        say(root->right);
    }
    return count;
}
```

inorder → 1, 5, 2, 10, 40, 15, 17, 20, 25, 30
post order → 1, 7, 5, 40, 10, 17, 30, 25, 20, 15

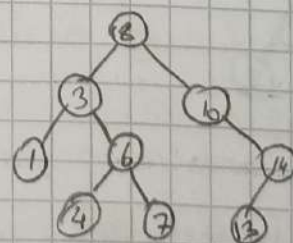


örnek

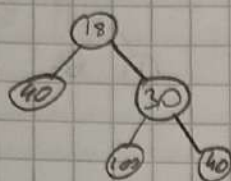
```
struct node *tree = newNode(8);
tree->left = newNode(6);
tree->right = newNode(10);
tree->left->left = newNode(2);
struct node *tmp = tree->left->left;
free(tree);
printf("%d", tmp->data);
```

Göster ama boş yere de.

inorder → 1, 3, 4, 6, 7, 8, 10, 13, 14
post order → 1, 4, 7, 6, 3, 13, 14, 10, 8



Inorder: 40, 18, 109, 30, 40
 preorder: 18, 40, 30, 109, 40



struct tree {

int data;
 struct node *left;
 struct node *right;
 };

struct node *createNode(int x) {

struct node *yeni = (struct node *) malloc(sizeof(struct node));

yeni->left = yeni->right = NULL;

yeni->data = x;

return yeni;

}

struct node *insert(struct node *root, int x) {

if (root == NULL)

return createNode(x);

if (x < root->data)

root->left = insert(root->left, x);

else

root->right = insert(root->right, x);

return root;

}

mainde

struct tree *yeni;

yeni = createNode(5);

yeni = insert(yeni, 4);

yeni = insert(yeni, 9);

örnek test et

struct node *search(struct node *root, int x) {

if (root->data == x)

return root;

else {

if (root->data > x)

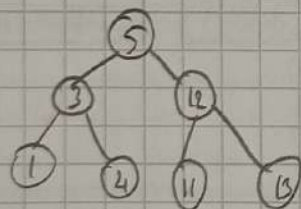
search(root->left, x);

else

search(root->right, x);

return root;

}



baş da test et

struct node *tmp2 = createNode(-1);

struct node *search(struct node *root, int x) {

if (root == NULL)

return tmp2;

if (root->data == x)

return root;

if (x < root->data)

return search(root->left, x);

else

return search(root->right, x);

}

mainde

struct node *tmp = search(yeni, 4);

if (tmp->data == 4)

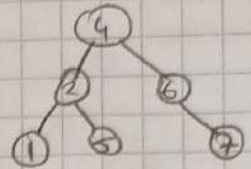
printf("Veri mevcut");

if (tmp->data == -1)

printf("Veri yok");

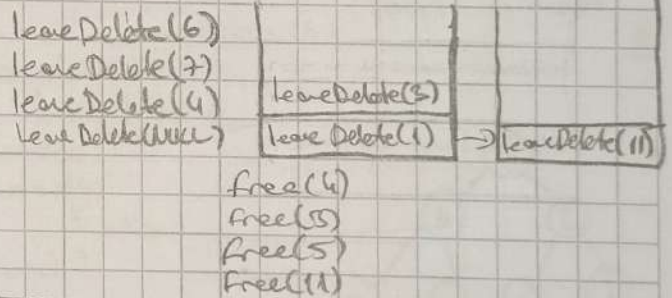
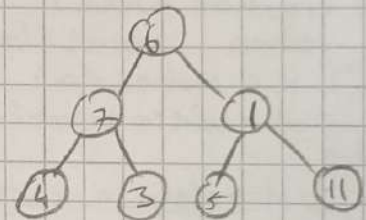
Binary Search Tree olup olmadığını kontrol etme

```
int check(struct node *root){
    if (root == NULL)
        return 1;
    if (root->left != NULL && root->left->data > root->data)
        return 0;
    if (root->right != NULL && root->right->data < root->data)
        return 0;
    if (!check(root->left) || !check(root->right))
        return 0;
    return 1;
}
```



Yaprak silme rekürsif olarak silme

```
struct tree *leaveDelete(struct tree *root){
    if (root->left == NULL && root->right == NULL){
        free(root);
        return NULL;
    }
    root->left = leaveDelete(root->left);
    root->right = leaveDelete(root->right);
    return root;
}
```



```
struct node *bul(struct node *root, int x){
    if (root == NULL)
        return NULL;
    if (root->data > x){
        struct node *tmp = bul(root->left, x);
        if (tmp == NULL)
            return root;
        else
            return tmp;
    }
    else
        return bul(root->right, x);
}
```

```
struct node *bul(struct node *root, int x){
    if (root == NULL || root->data == x)
        return root;
    if (x < root->data)
        return bul(root->left, x);
    else
        return bul(root->right, x);
}
```

Gelişir gibi

solama gib: dınyar

GRAPH

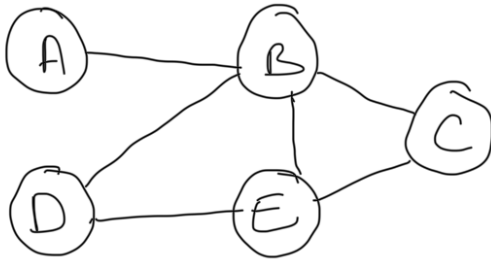
Graph sadeleşmiş bir veri yapısı uygulamasıdır

Google maps

Network vb

$$G = (V, E)$$

↓ ↘
Vertex Edge

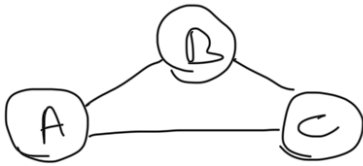


$$V = \{A, B, C, D, E\}$$

$$E = (A, B), (B, C), (C, E), (D, E), (D, B), (A, D)$$

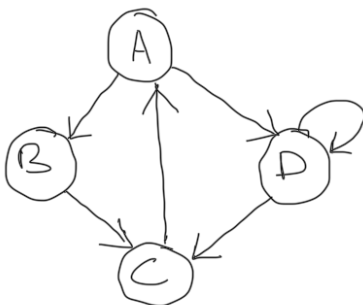
UNDIRECTED GRAPH

- Yönlendirme olmayan graph eğer A-B arasında kenar varsa B-A arasında da vardır

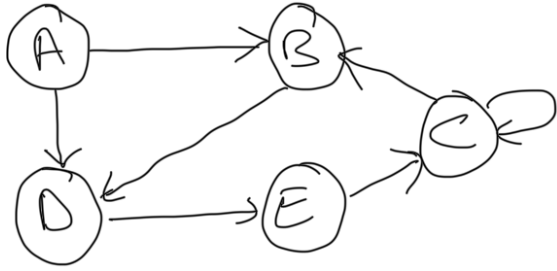


DIRECTED GRAPH

- Her kenar bağlantısı yönlendirilmiş graph.



Degree $\rightarrow \deg(v) = 0$

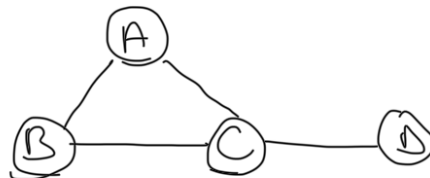
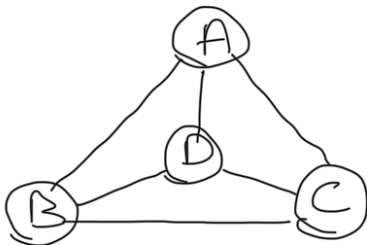


$$\deg(C)_D = 4$$

Kaçtane ok girmiş yada çıkmışsa degree sayısına eşittir ~~ok~~ kendisinden çıkıp kendisini işaret ediyorsa 2 kez sayılır.

REGULAR GRAPH

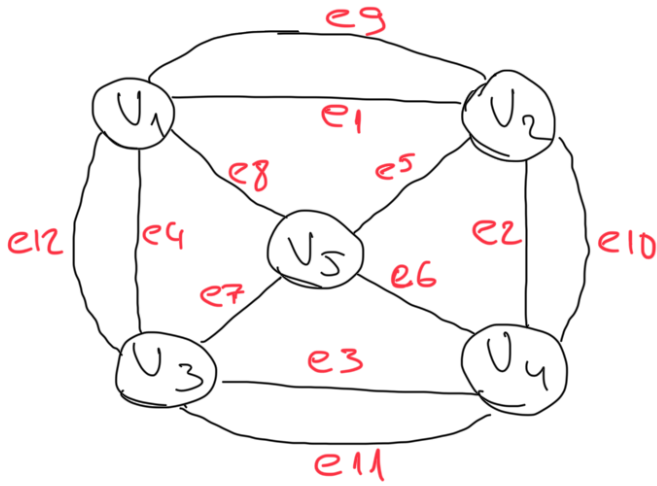
Her düğümün eşit sayıda komşu düğümü olan graph.



$(A, B, C, D) \rightarrow \text{open walk}$

walk : Sonlu sayıda düğüm ve kenar geçişleri

open walk : Kenarların tekrar edilmediği geçişler.



Closed walk: Başlangıç ve bitiş düğümleri aynı olmasına denir.

$V_1 e_1 V_2 e_2 V_4 e_{11} V_3 e_4 V_1 \rightarrow$ closed walk

Path: Gezintide sadece düğümler yazılır.

closed path: Başlangıç ve bitiş düğümleri aynı olmasına denir.

SIMPLE PATH

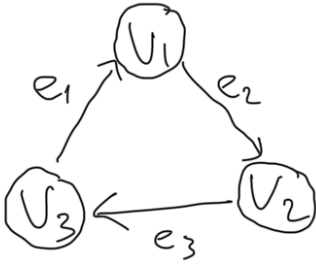
yoldaki tüm düğümlerin eşsiz olma durumudur

A, B, C ✓

A, B, C, A ✗

CYCLE

Yol için hiçbir düğüm ve kenarın tekrar edilmemesidir. Başlangıç ve Bitiş düğümleri hariç



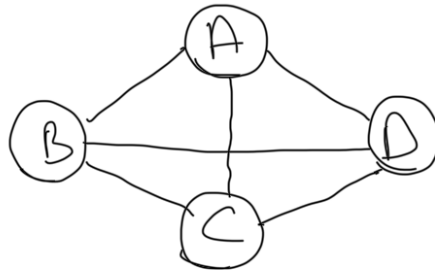
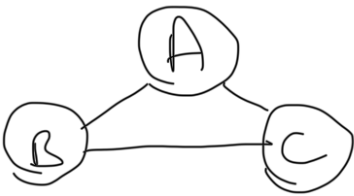
$V_1 e_2 V_2 e_3 V_3 e_1 V_1 \rightarrow \text{cycle}$

COMPLETE GRAPH

Bir düğümünden diğer tüm düğümlere bağlantı olan graph

$N \rightarrow$ düğüm sayısı

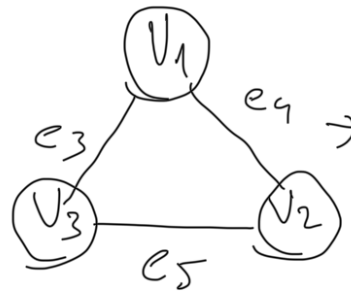
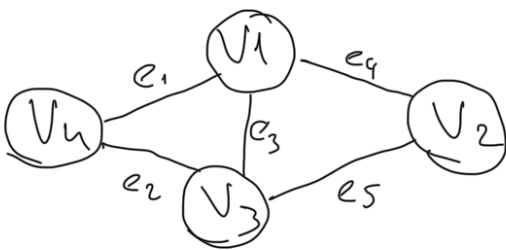
$N(N-1)/2$ kenar sayısı



Clique (undirected graph)

$G = (V, E)$, alt seti V 'nin

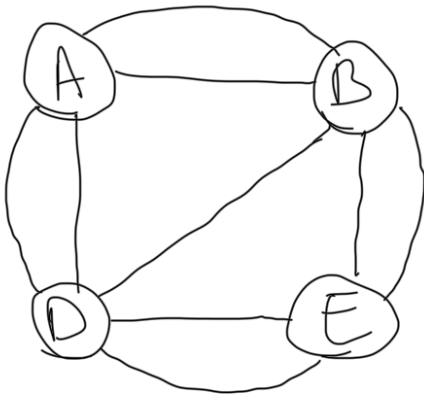
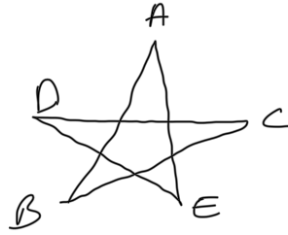
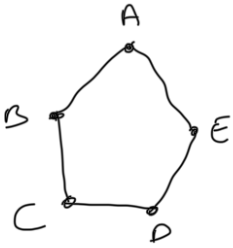
Her iki düğümün bağlantısı olmalı



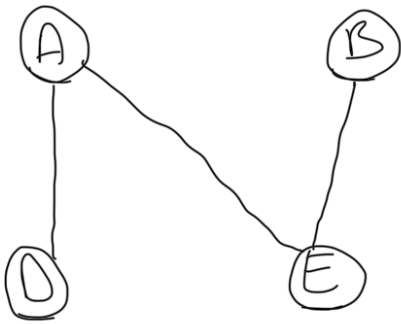
Bu bir
Clique'dir

Weighted Graph

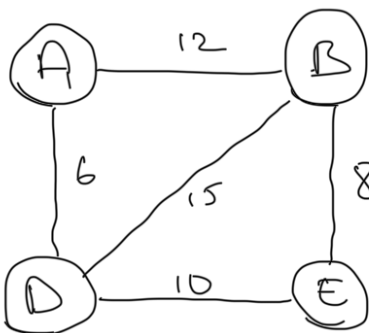
Her kenar için maliyet vardır.



→ multi graph



→ Tree



→ weighted graph

OPERASYONLAR

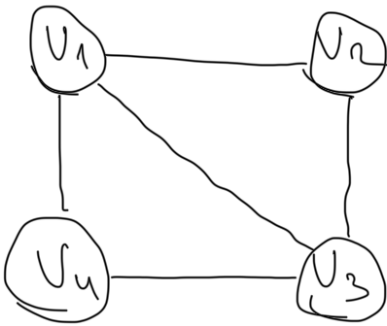
Create → Bellekte tutma

Traverse → Her düğümü 1 kez gez

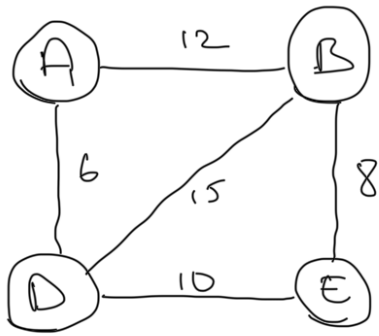
insertion → Ekleme

Deletion → Düğümü sil

Searching → Düğüm ara



	V ₁	V ₂	V ₃	V ₄
V ₁	0	1	1	1
V ₂	1	0	1	0
V ₃	1	1	0	1
V ₄	1	0	1	0



	A	B	D	E
A		12	6	5
B	12			8
D	6			10
E	5	8	10	

graph oluşturma koduna bakalım

// Print, yazdırma //